

Unified Engineering

16.002 Signals and Systems

Review of Solving Ordinary Differential Equations

Jonathan P. How
Massachusetts Institute of Technology
jhow@mit.edu

The goal is to discuss a simple solution methodology for ordinary differential equations – these are assumed to be linear and the derivatives are only with respect to one variable. So

$$\frac{dy(t)}{dt} + y(t) = 3$$

is an ODE, but

$$\frac{dy(t)}{dt} + y^3(t) = 3$$

isn't (it is nonlinear), and neither is (function of more than one variable)

$$\frac{d^2y(x, w)}{dx^2} + \frac{d^2y(x, w)}{dw^2} = 3.$$

Writing $\dot{y}(t) \equiv \frac{dy(t)}{dt}$, the basic form of an ODE is:

$$y^{(n)}(t) + a_1 y^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}(t) + a_{n-1} \dot{y}(t) + a_n y(t) = cw(t)$$

where $w(t)$ is the input that is often constant, but can be a function of time, and the coefficients a_i and c are constant.

1.1 Method of Assumed Solutions

The solution approach is to **assume the form of an answer and then verify that it is the solution** (method of assumed solutions), which typically involves finding the unknown constants in the assumed form of the solution to ensure that the ODE is satisfied. This solution process must be executed in two steps.

1. The **first** is to split the answer into two parts – the homogeneous part $y_h(t)$ and the forced part $y_f(t)$, so that

$$y(t) = y_h(t) + y_f(t)$$

where, by definition, the homogeneous part solves the equation

$$y_h^{(n)}(t) + a_1 y_h^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}_h(t) + a_{n-1} \dot{y}_h(t) + a_n y_h(t) = 0$$

and we select $y_f(t)$ to satisfy

$$y_f^{(n)}(t) + a_1 y_f^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}_f(t) + a_{n-1} \dot{y}_f(t) + a_n y_f(t) = cw(t).$$

Note that if we sum these two equations,

$$\begin{aligned} & y_h^{(n)}(t) + a_1 y_h^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}_h(t) + a_{n-1} \dot{y}_h(t) + a_n y_h(t) \\ & + \left(y_f^{(n)}(t) + a_1 y_f^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}_f(t) + a_{n-1} \dot{y}_f(t) + a_n y_f(t) \right) \\ & = y^{(n)}(t) + a_1 y^{(n-1)}(t) + \dots + a_{n-2} \ddot{y}(t) + a_{n-1} \dot{y}(t) + a_n y(t) = cw(t) \end{aligned}$$

so that $y(t) = y_h(t) + y_f(t)$ is a solution to the original ODE.

2. The **second** step is to assume a particular form for $y_h(t)$ (usually an exponential)

$$y_h(t) = Ce^{pt}$$

where the constants C and p have to be found to ensure that the ODE is satisfied.

Note: that initial conditions specified for the problem must be applied to the complete $y(t)$.

1.2 First Order Systems

Specializing to a first order ODE (one derivative)

$$\dot{y}(t) + ay(t) = cw(t)$$

and substituting $y_h(t)$ gives

$$Cpe^{pt} + aCe^{pt} = 0$$

where the right hand side is zero because this is the homogeneous solution. Note that the Ce^{pt} terms can be canceled out, leaving $p + a = 0$, so we must have that $p = -a$ to satisfy this ODE. Thus we know that

$$y_h(t) = Ce^{-at}.$$

Had the input been constant $w(t) = B$, then we know that $y_f(t) = cB/a$ is a possible solution (all derivatives of this $y_f(t)$ are zero) and thus

$$y(t) = y_h(t) + y_f(t) = \frac{cB}{a} + Ce^{-at}$$

where the coefficient C can now be found by ensuring that the initial condition is satisfied.

For example, if we had been told that $y(0) = 0$, then $y(0) = \frac{cB}{a} + Ce^{-a0} = \frac{cB}{a} + C = 0$, which means that $C = -\frac{cB}{a}$ and

$$y(t) = y_h(t) + y_f(t) = \frac{cB}{a}(1 - e^{-at})$$

This solution makes sense (and satisfies the ODE) because $y(0) = 0$ and $\lim_{t \rightarrow \infty} y(t) = cB/a$, which is the forced solution.

1.2.1 Useful guess table for forced solutions

For many common inputs, the method of assumed solutions starts by trying one of the forms below for the forced response $y_f(t)$:

Input $w(t)$	Try this form for $y_f(t)$
Constant B	C
$e^{\beta t}$	$Ce^{\beta t}$
$\cos(\omega t)$ or $\sin(\omega t)$	$C_1 \cos(\omega t) + C_2 \sin(\omega t)$
Polynomial in t of degree m	polynomial in t of degree m

If the input is a sum of terms, use a sum of corresponding trial forms for $y_f(t)$.

Resonance rule (repeated-root fix)

If your trial $y_f(t)$ contains any term already present in $y_h(t)$, the trial is not linearly independent and will fail. In that case, multiply the trial by t enough times to make it independent:

$$y_f^{\text{new}}(t) = t^r y_f^{\text{trial}}(t)$$

where r is the multiplicity of the overlap with the homogeneous solution. For example, if e^{-3t} already appears in $y_h(t)$, then try te^{-3t} ; if te^{-3t} also appears, then try t^2e^{-3t} .

Example 1: (first order, constant input) If $y(0) = 4$, solve for $y(t)$ given

$$\dot{y}(t) + 3y(t) = 9$$

Here we know that $y_h(t) = Ce^{pt}$ would yield

$$Cpe^{pt} + 3Ce^{pt} = 0 \quad \rightarrow \quad p = -3$$

and $y_h(t) = Ce^{-3t}$. Further we can guess that $y_f(t) = 9/3 = 3$, yielding

$$y(t) = y_h(t) + y_f(t) = 3 + Ce^{-3t}$$

To impose the initial condition, let $y(0) = 3 + C = 4$, so $C = 1$ and $y(t) = 3 + e^{-3t}$.

Example 2: (first order, exp input) Consider the same system with different input $w(t) = e^{\beta t}$. We can try an assumed forced response of the form $y_f(t) = Ce^{\beta t}$ to get

$$\dot{y}_f(t) + 3y_f(t) = e^{\beta t} \quad \rightarrow \quad C\beta e^{\beta t} + 3Ce^{\beta t} = e^{\beta t}$$

which will be true if $C(\beta+3) = 1$ or $C = 1/(\beta+3)$, which works provided that $\beta \neq -3$, because if it did, then this forced solution satisfies the homogeneous ODE and would give zero on the LHS resulting in a contradiction. Here the overall solution is

$$y(t) = y_h(t) + y_f(t) = Ce^{-3t} + \frac{e^{\beta t}}{3 + \beta}$$

where $y(0) = 4$ would mean that $C + \frac{1}{3 + \beta} = 4$ and

$$y(t) = (4 - \frac{1}{3 + \beta})e^{-3t} + \frac{1}{3 + \beta}e^{\beta t}$$

Example 3: (first order, repeated roots) If the input was actually of the form $w(t) = e^{-3t}$, then we would have to assume a different forced solution of the form $y_f(t) = C_2 te^{\beta t}$ yielding

$$\dot{y}_f(t) + 3y_f(t) = e^{-3t} \quad \rightarrow \quad -3C_2 te^{-3t} + C_2 e^{-3t} + 3C_2 te^{-3t} = e^{-3t}$$

Since the first and last terms of the lefthand side cancel, we need $C_2 = 1$ to satisfy the equation. The combined solution would now be

$$y(t) = y_h(t) + y_f(t) = Ce^{-3t} + te^{-3t}$$

where $y(0) = 4$ would mean that $C = 4$ and $y(t) = (4 + t)e^{-3t}$.

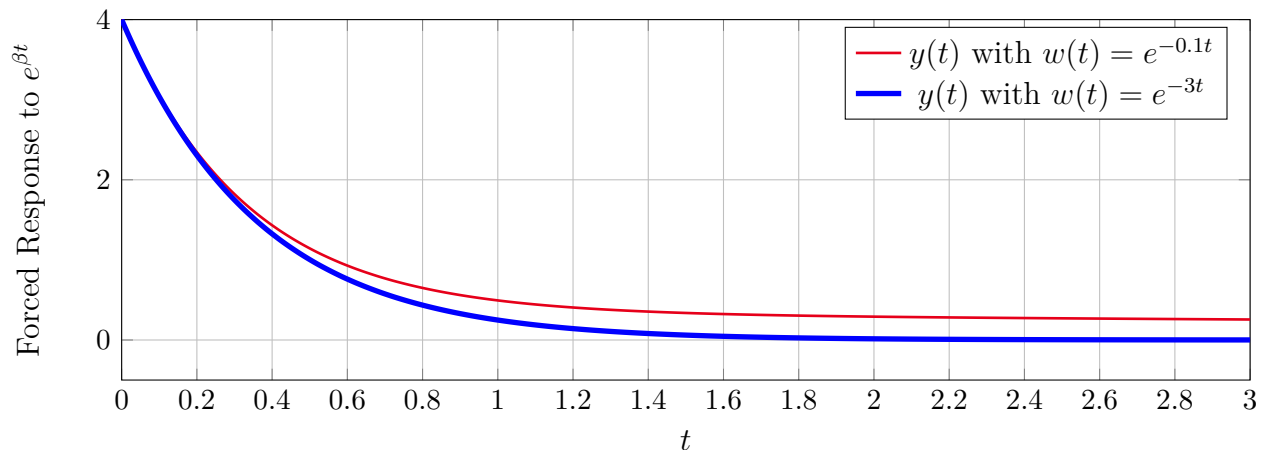


Figure 1.1: Comparison of two first order system responses to different exponential inputs.

1.2.2 Sinusoidal Inputs

Consider the same first order system with a different input $w(t) = \cos(\omega t)$. We can try an assumed forced response of the form $y_f(t) = C \cos(\omega t)$ but we might anticipate some issues since $\dot{y}$ will be $-C\omega \sin(\omega t)$ and so the functional forms don't cancel out nicely.

More generally this can be done for a complex input $w(t) = e^{i\omega t}$ and then we take the real or imaginary part of the solution depending on the type of input we want to consider (cos or sin). Note that now we have to assume that $y_f(t) = Ce^{i\omega t}$, i.e. that the output is some scaled version of the input (seems reasonable as the system is linear).

$$y_f(t) = Ce^{i\omega t}, \quad \dot{y}_f(t) = i\omega Ce^{i\omega t}$$

so we have that

$$\dot{y}_f(t) + 3y_f(t) = e^{i\omega t} \quad \rightarrow \quad i\omega Ce^{i\omega t} + 3Ce^{i\omega t} = e^{i\omega t}$$

requiring that $C(3 + i\omega) = 1$ or $C = \frac{1}{(3 + i\omega)} = \frac{1}{\sqrt{9 + \omega^2}}e^{i\phi}$ where to find ϕ note that $C = \frac{1}{(3 + i\omega)} = \frac{3 - i\omega}{9 + \omega^2}$, so $\tan(\phi) = \frac{-\omega}{3}$.

Example 4: (first order, sinusoidal input) If the input happened to be $w(t) = \cos(3t)$ so that $\omega = 3$, the forced output would be the real part of the signal computed above $y_f(t) = \text{real} \left(\frac{1}{\sqrt{9 + 9}}e^{i\phi} \right)$ with $\phi = \arctan \left(\frac{-3}{3} \right) = -\frac{\pi}{4}$, so that

$$y_f(t) = \frac{1}{\sqrt{18}} \cos \left(3t - \frac{\pi}{4} \right)$$

the total solution would be

$$y(t) = y_h(t) + y_f(t) = Ce^{-3t} + \frac{1}{\sqrt{18}} \cos \left(3t - \frac{\pi}{4} \right)$$

and $y(0) = 4$ implies that $C + \frac{1}{\sqrt{18}} \frac{1}{\sqrt{2}} = C + \frac{1}{\sqrt{36}} = 4$ or $C = 4 - \frac{1}{6}$. Substituting gives

$$y(t) = \frac{23}{6}e^{-3t} + \frac{\sqrt{2}}{6} \cos \left(3t - \frac{\pi}{4} \right)$$

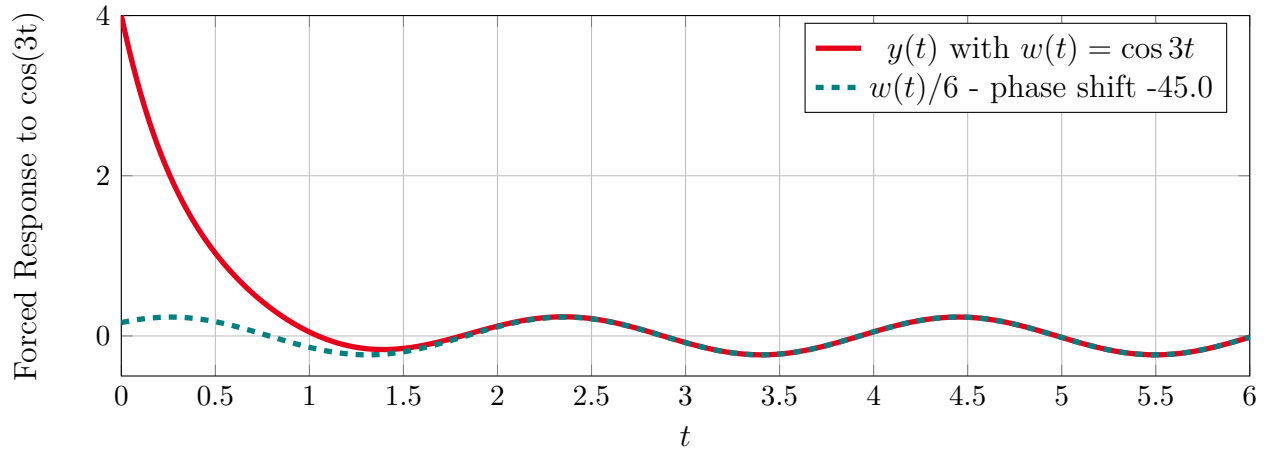


Figure 1.2: First order response to sinusoidal input showing rapid initial decay to a shifted sinusoid. Shifted input provided as a reference.

1.3 Higher Order Systems

For higher order systems, the method of assumed solutions gets a bit more complicated as we end up with higher order equations to solve for p . This is tractable for second order ODE (solve a quadratic), but requires a computer for higher order.

If the roots of the quadratic are real p_1 and p_2 , but not repeating, then the homogeneous solution will be of the form

$$y_h(t) = C_1 e^{p_1 t} + C_2 e^{p_2 t} \quad (1)$$

which is combined with the forced solution to obtain the final result. Had the poles been repeated, then we would have to consider homogeneous solutions of the form

$$y_h(t) = (C_1 + C_2 t) e^{pt} \quad (2)$$

giving

$$\dot{y}_h(t) = C_2 e^{pt} + p(C_1 + C_2 t) e^{pt}, \quad (3)$$

$$\ddot{y}_h(t) = p C_2 e^{pt} + p(C_2) e^{pt} + p^2(C_1 + C_2 t) e^{pt} \quad (4)$$

Example 5: (real, non-repeating poles; constant input) solve

$$\ddot{y}(t) + 3\dot{y}(t) + 2y(t) = 1$$

for positive t given initial conditions (IC) that $y(0) = 1$ and $\dot{y}(0) = 0$. Here assuming that $y_h(t) = Ce^{pt}$, substitution gives

$$p^2Ce^{pt} + 3pCe^{pt} + 2Ce^{pt} = 0$$

requiring that $p^2 + 3p + 2 = 0$ with roots $p = -1, -2$ so

$$y_h(t) = C_1e^{-t} + C_2e^{-2t} \quad (5)$$

The forced solution for a constant input is guessed to be $y_f(t) = 1/2$, so

$$y(t) = 0.5 + C_1e^{-t} + C_2e^{-2t} \quad (6)$$

Impose the IC to get $y(0) = 1 = 0.5 + C_1 + C_2$ and $\dot{y}(0) = 0 = -C_1 - 2C_2$. So $C_1 = -2C_2$ and $-C_2 = 0.5$, $C_1 = 1$. The final result is that

$$y(t) = 0.5 + e^{-t} - 0.5e^{-2t}$$

Example 6: (real, repeating poles; constant input) solve

$$\ddot{y}(t) + 2\dot{y}(t) + 1y(t) = 1$$

(repeated root at $p = -1$) for positive t given initial conditions (IC) that $y(0) = 1$ and $\dot{y}(0) = 1$. Here assuming that $y_h(t) = (C_1 + C_2t)e^{pt}$, with $p = -1$ substitution gives

$$-2C_2e^{-t} + (C_1 + C_2t)e^{-t} + 2(C_2e^{-t} - (C_1 + C_2t)e^{-t}) + (C_1 + C_2t)e^{-t} = 0$$

divide through

$$-2C_2 + (C_1 + C_2t) + 2(C_2 - (C_1 + C_2t)) + (C_1 + C_2t) = 0$$

giving

$$C_1(-2 + 1 + 1) + C_2(-2 + t + 2 - 2t + t) = 0$$

so the solution does satisfy the ODE. The initial conditions can then be used to determine C_1 and C_2 using the total solution

$$y(t) = 1 + (C_1 + C_2t)e^{-t} \quad (7)$$

Impose the IC to get $y(0) = 1 = 1 + C_1$ so $C_1 = 0$ and $\dot{y}(0) = 1 = C_2$. The final result is that

$$y(t) = 1 + te^{-t}$$

Example 7: (real, non-repeating poles; exp input) Solve

$$\ddot{y}(t) + 3\dot{y}(t) + 2y(t) = e^{\beta t}$$

for positive t given initial conditions (IC) that $y(0) = 1$ and $\dot{y}(0) = 0$. The homogeneous solution is as above, but the forced solution is now $y_f(t) = Ce^{\beta t}$ where we assume that $\beta \neq -1, -2$. Substitute to get

$$C\beta^2 e^{\beta t} + 3\beta C e^{\beta t} + 2C e^{\beta t} = e^{\beta t}$$

which requires that $C = \frac{1}{\beta^2 + 3\beta + 2}$. The full solution is then

$$y(t) = C_1 e^{-t} + C_2 e^{-2t} + \frac{e^{\beta t}}{\beta^2 + 3\beta + 2} \quad (8)$$

Impose the IC to get $y(0) = 1 = C_1 + C_2 + \frac{1}{\beta^2 + 3\beta + 2}$ and $\dot{y}(0) = 0 = -C_1 - 2C_2 + \frac{\beta}{\beta^2 + 3\beta + 2}$. So $C_1 = -2C_2 + \frac{\beta}{\beta^2 + 3\beta + 2}$ and

$$\frac{\beta^2 + 3\beta + 1}{\beta^2 + 3\beta + 2} = -C_2 + \frac{\beta}{\beta^2 + 3\beta + 2}$$

so

$$C_2 = -\frac{\beta^2 + 2\beta + 1}{\beta^2 + 3\beta + 2}, \quad C_1 = 2\frac{\beta^2 + 2\beta + 1}{\beta^2 + 3\beta + 2} + \frac{\beta}{\beta^2 + 3\beta + 2} = \frac{2\beta^2 + 5\beta + 2}{\beta^2 + 3\beta + 2}$$

The final result is that

$$y(t) = \frac{(2\beta^2 + 5\beta + 2)e^{-t} - (\beta^2 + 2\beta + 1)e^{-2t} + e^{\beta t}}{(\beta + 1)(\beta + 2)}$$

which simplifies a bit because $2\beta^2 + 5\beta + 2 = (\beta + 1)(\beta + 2)$ and $\beta^2 + 2\beta + 1 = (\beta + 1)(\beta + 1)$

$$y(t) = \frac{(\beta + 1)e^{-t}}{(\beta + 1)} - \frac{(\beta + 1)e^{-2t}}{\beta + 2} + \frac{e^{\beta t}}{(\beta + 1)(\beta + 2)}$$

1.3.1 Complex Roots

Further, for second order, one can end up with complex solutions for p , implying that sinusoids are involved since we know that

$$e^{i\theta} = \cos \theta + i \sin \theta$$

meaning that

$$\cos \theta = \frac{e^{i\theta} + e^{-i\theta}}{2} \quad \text{and} \quad \sin \theta = \frac{e^{i\theta} - e^{-i\theta}}{2i}$$

Thus if we substitute the assumed solution into a second order ODE and solve for p to obtain a complex solution pair $p = p_r \pm ip_i$ (this will not always be the case), then

$$y_h(t) = C_1 e^{(p_r + ip_i)t} + C_2 e^{(p_r - ip_i)t} = e^{p_r t} (C_1 e^{ip_i t} + C_2 e^{-ip_i t}) \quad (9)$$

$$= e^{p_r t} \left[(C_1 + C_2) \frac{(e^{ip_i t} + e^{-ip_i t})}{2} + i(C_1 - C_2) \frac{(e^{ip_i t} - e^{-ip_i t})}{2i} \right] \quad (10)$$

$$= e^{p_r t} [(C_1 + C_2) \cos(p_i t) + i(C_1 - C_2) \sin(p_i t)] \quad (11)$$

This second last step looks a bit ugly, but typically $C_2 = C_1^*$ (they are complex conjugates of each other), and then $C_1 + C_2 = 2\text{real}(C_1)$ and $i(C_1 - C_2) = -2\text{imag}(C_1)$, so things simplify a bit.

Example 8: (complex roots, constant input) solve

$$\ddot{y}(t) + 2\dot{y}(t) + 10y(t) = 1$$

for positive t given initial conditions (IC) that $y(0) = 1$ and $\dot{y}(0) = 0$. Start with the homogeneous part: $y_h(t) = Ce^{pt}$ - substitute and cancel to get

$$p^2 + 2p + 10 = 0$$

with roots $p = -1 \pm 3i$ so that

$$y_h(t) = C_1 e^{(-1+3i)t} + C_2 e^{(-1-3i)t}$$

Now look at the forced part: Since the input is a constant, we can guess that the forced response will also be a constant (all derivatives are zero), so that $10y_f(t) = 1$, or $y_f(t) = 0.1$. Thus we have

$$y(t) = 0.1 + C_1 e^{(-1+3i)t} + C_2 e^{(-1-3i)t}$$

Now we need to impose the IC:

$$y(0) = 0.1 + C_1 e^0 + C_2 e^0 = 1 \rightarrow C_1 + C_2 = 0.9 \quad (12)$$

$$\dot{y}(0) = C_1(-1 + 3i)e^0 + C_2(-1 - 3i)e^0 = 0 \rightarrow C_1 = C_2 \frac{(1 + 3i)}{-1 + 3i} \quad (13)$$

giving 2 equations in 2 unknowns. Solving gives

$$C_2(1 + \frac{1 + 3i}{-1 + 3i}) = C_2(\frac{6i}{-1 + 3i}) = 0.9$$

so $C_2 = 0.9 \frac{-1 + 3i}{6i} = \frac{0.9(3 + i)}{6}$ and $C_1 = \frac{0.9(3 - i)}{6}$. Since in the case $p_r = -1$ and $p_i = 3$, we get the simpler version of the response as

$$y(t) = 0.1 + C_1 e^{(-1+3i)t} + C_2 e^{(-1-3i)t} \quad (14)$$

$$= 0.1 + \frac{0.9(3 - i)}{6} e^{(-1+3i)t} + \frac{0.9(3 + i)}{6} e^{(-1-3i)t} \quad (15)$$

$$= 0.1 + e^{-t}(0.9 \cos(3t) + \frac{1.8}{6} \sin(3t)) \quad (16)$$

Confirming that $y(0) = 0.1 + 0.9 = 1$ and

$$\dot{y}(t) = -e^{-t}(0.9 \cos(3t) + \frac{1.8}{6} \sin(3t)) + e^{-t}(-3 \cdot 0.9 \sin(3t) + 3 \frac{1.8}{6} \cos(3t))$$

so $\dot{y}(0) = -0.9 + 1.8/2 = 0$ as required. We can simplify further by noting that

$$y(t) = 0.1 + 0.9e^{-t}(\cos(3t) + \frac{1}{3} \sin(3t)) = 0.1 + 0.9\sqrt{1 + (1/3)^2}e^{-t} \cos(3t + \phi) \quad (17)$$

where $\tan \phi = -1/3$.

Example 9: (complex roots, sinusoidal input) Note that other forced inputs can also be considered, such as a sinusoid, so that $w(t) = \cos(\omega t)$. For this example,

$$\ddot{y}(t) + 2\dot{y}(t) + 10y(t) = \cos(\omega t)$$

More generally this can be done for a complex input $w(t) = e^{i\omega t}$ and then we take the real or imaginary part of the solution depending on the type of input we want to consider (cos or sin). Note that now we have to assume that $y_f(t) = Ce^{i\omega t}$, i.e. that the output is some scaled version of the input (seems reasonable as the system is linear).

$$y_f(t) = Ce^{i\omega t}, \quad \dot{y}_f(t) = i\omega Ce^{i\omega t}, \quad \ddot{y}_f(t) = -\omega^2 Ce^{i\omega t}$$

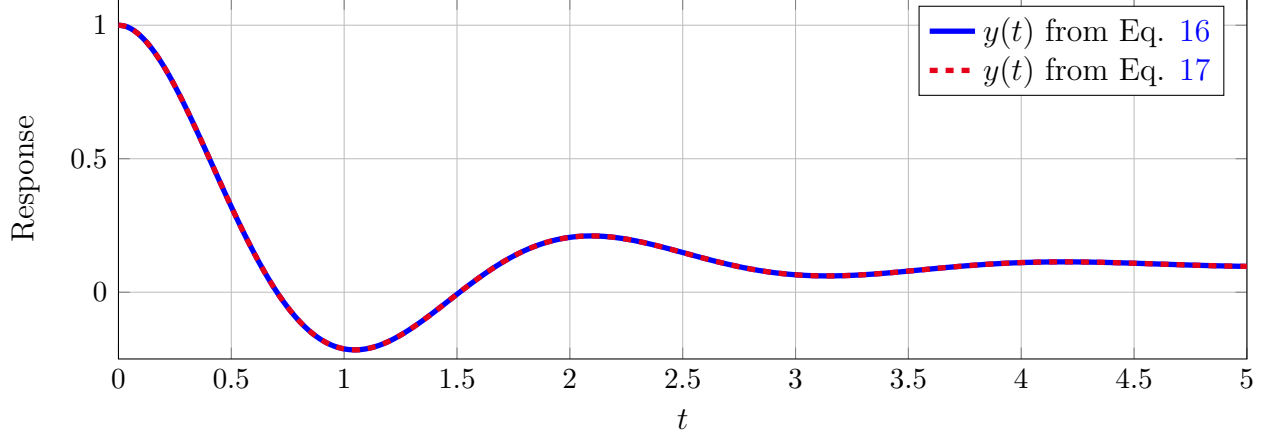


Figure 1.3: Response of second order system with complex roots. Compares two equivalent representations of the same solution.

so we have that

$$-\omega^2 C e^{i\omega t} + 2i\omega C e^{i\omega t} + 10C e^{i\omega t} = e^{i\omega t}$$

which can only be true if

$$C(10 - \omega^2 + 2\omega i) = 1 \quad \rightarrow \quad C = \frac{1}{10 - \omega^2 + 2\omega i} = \frac{10 - \omega^2 - 2\omega i}{(10 - \omega^2)^2 + 4\omega^2} = C_r + iC_i$$

so the coefficient is complex. Thus the forced response is

$$y_f(t) = C e^{i\omega t} = (C_r + iC_i)(\cos(\omega t) + i \sin(\omega t)) \quad (18)$$

$$= (C_r \cos(\omega t) - C_i \sin(\omega t)) + i(C_r \sin(\omega t) + C_i \cos(\omega t)) \quad (19)$$

where both $C_r = (10 - \omega^2)/((10 - \omega^2)^2 + 4\omega^2)$ and $C_i = -2\omega/((10 - \omega^2)^2 + 4\omega^2)$ depend on the input frequency ω . So if the input was $w(t) = (\cos \omega t)$, then we just take the real part of the forced output

$$y_f(t) = \frac{(10 - \omega^2)}{((10 - \omega^2)^2 + 4\omega^2)} \cos(\omega t) + \frac{2\omega}{((10 - \omega^2)^2 + 4\omega^2)} \sin(\omega t) \quad (20)$$

$$= \frac{(10 - \omega^2) \cos(\omega t) + 2\omega \sin(\omega t)}{((10 - \omega^2)^2 + 4\omega^2)} \quad (21)$$

$$= \frac{\cos(\omega t + \phi)}{\sqrt{(10 - \omega^2)^2 + 4\omega^2}} \quad (22)$$

where $\tan \phi = -\frac{2\omega}{(10 - \omega^2)}$.

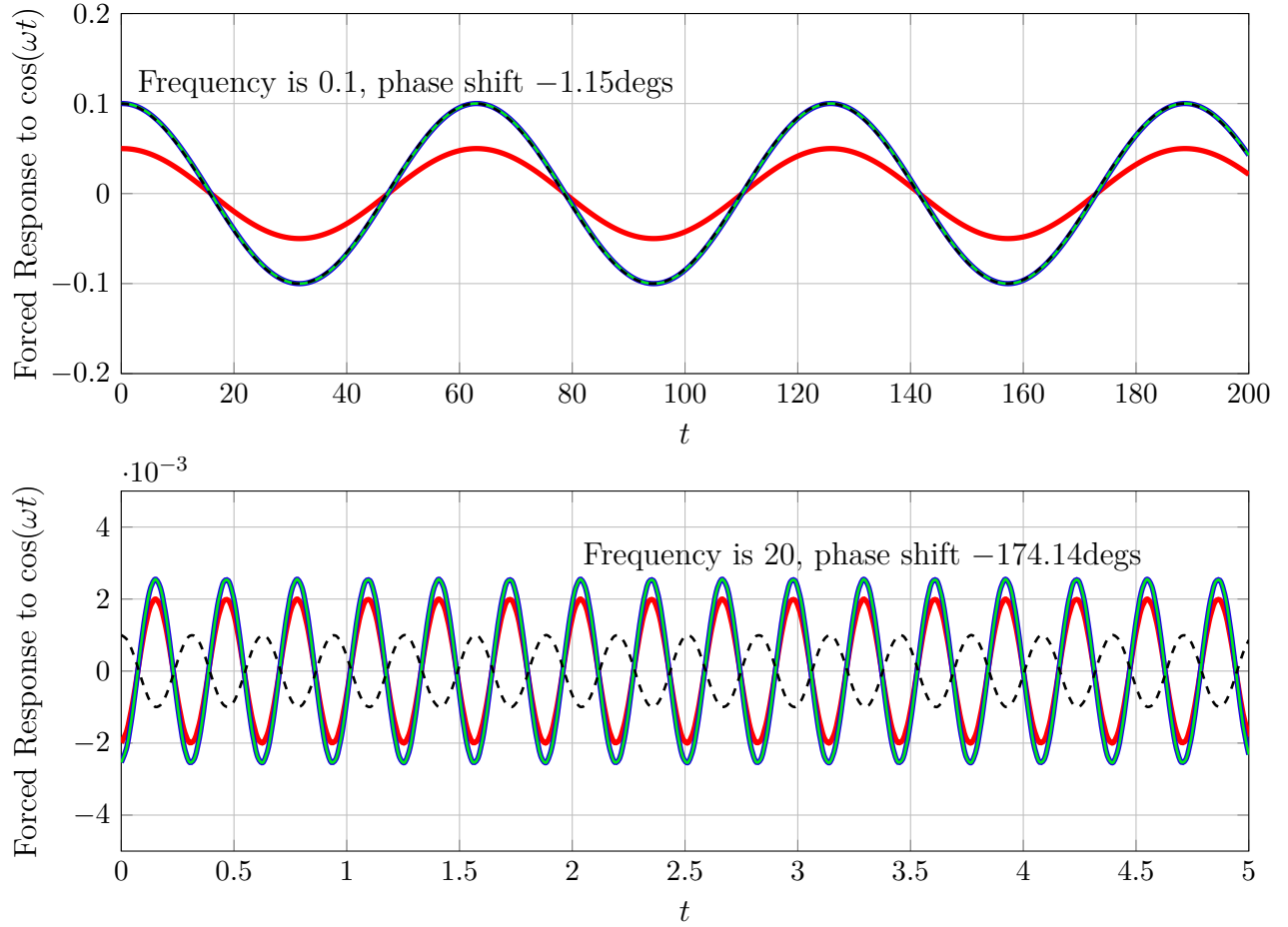


Figure 1.4: Comparison of 2 forms of the forced response (only) (Equations 20 and 22) for different frequency inputs. response in blue – the input is in black, and the shifted (and scaled) input shown in red to show that the phase prediction (not amplitude) is accurate.

As before, the total response is

$$y(t) = y_h(t) + y_f(t) = C_1 e^{(-1+3i)t} + C_2 e^{(-1-3i)t} + \frac{(10 - \omega^2) \cos(\omega t) + 2\omega \sin(\omega t)}{\omega^4 - 16\omega^2 + 100}$$

so $y(0) = 1$ means that

$$1 = C_1 + C_2 + \frac{(10 - \omega^2)}{\omega^4 - 16\omega^2 + 100}$$

or

$$C_1 + C_2 = \frac{\omega^4 - 15\omega^2 + 90}{\omega^4 - 16\omega^2 + 100}$$

Also

$$\dot{y}(0) = C_1(-1 + 3i) + C_2(-1 - 3i) + \frac{2\omega^2}{\omega^4 - 16\omega^2 + 100} = 0$$

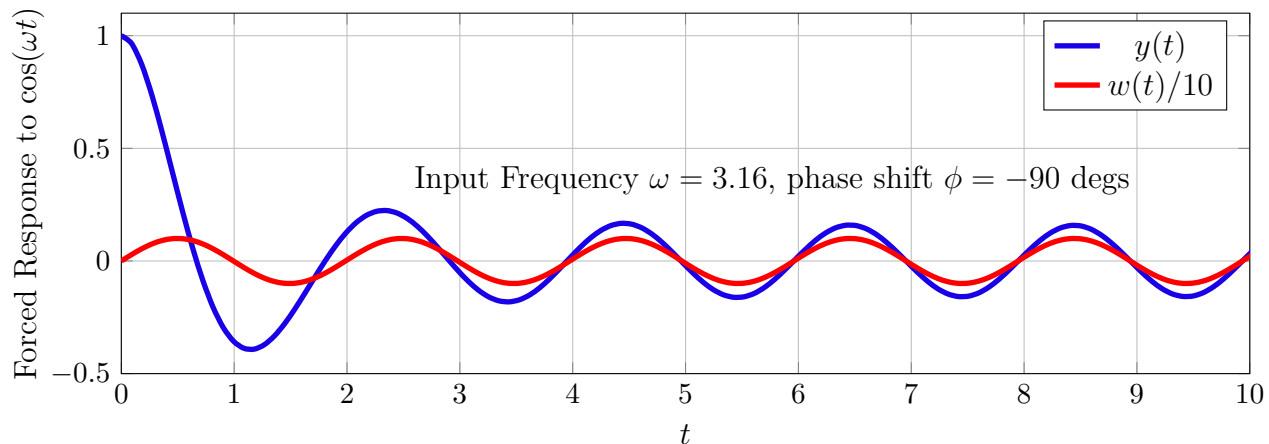


Figure 1.5: Total response to sinusoidal input. Red line shows phase shifted (and scaled) input for comparison.

This solves to give

$$C_1 = \frac{3(\omega^4 - 15\omega^2 + 90) - j(\omega^4 - 17\omega^2 + 90)}{6(\omega^4 - 16\omega^2 + 100)}$$

and $C_2 = C_1^*$. Using the expressions in (3) to clean up the homogeneous part we get

$$y(t) = e^{-t} \left(\frac{3(\omega^4 - 15\omega^2 + 90) \cos(3t) + (\omega^4 - 17\omega^2 + 90) \sin(3t)}{3(\omega^4 - 16\omega^2 + 100)} \right) + \frac{(10 - \omega^2) \cos(\omega t) + 2\omega \sin(\omega t)}{\omega^4 - 16\omega^2 + 100}$$

1.4 Solving ODEs in Python

There are different solution approaches that one can use in Python to solve ODEs.

Shared utilities: Common helper functions are defined here.

scripts/ode_common.py

```
1 from pathlib import Path
2
3 import numpy as np
4 import sympy as sym
5
6
7 t = sym.symbols("t")
8 y = sym.symbols("y", cls=sym.Function)(t)
9
10
11 def solve_ode(A=None, w=0, Y0=None):
12     """Return a callable solution for first- or second-order linear ODEs."""
13     if A is None:
14         A = [1, 1, 1]
15     if Y0 is None:
16         Y0 = [0, 0]
17
18     if len(A) == 3:
19         ode = A[0] * y.diff(t, t) + A[1] * y.diff(t) + A[2] * y - w
20         sol = sym.dsolve(
21             ode,
22             ics={y.subs(t, 0): Y0[0], sym.diff(y, t).subs(t, 0): Y0[1]},
23         )
24     else:
25         ode = A[0] * y.diff(t) + A[1] * y - w
26         sol = sym.dsolve(ode, ics={y.subs(t, 0): Y0[0]})
27
28     return sym.lambdify([t], sol.rhs, modules=["numpy"])
29
30
31 def default_inputs(phi=0.0):
32     we = 3 * sym.exp(-3 * t)
33     wc = 3 * sym.cos(3 * t + phi)
34     return we, wc
35
36
37 def output_dir():
38     out = Path(__file__).resolve().parent.parent / "figs"
39     out.mkdir(parents=True, exist_ok=True)
40     return out
```

Symbolic: The plotted symbolic/analytic cases are generated by these standalone scripts.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from ode_common import default_inputs, output_dir, solve_ode
5
6
7 def main():
8     phi = 0.0
9     we, wc = default_inputs(phi)
10    Y0 = [4]
11    A = [1, 3]
12
13    fun_0 = solve_ode(A=A, w=0, Y0=Y0)
14    fun_1 = solve_ode(A=A, w=9, Y0=Y0)
15    fun_2 = solve_ode(A=A, w=we, Y0=Y0)
16    fun_3 = solve_ode(A=A, w=wc, Y0=Y0)
17
18    tt = np.arange(0, 6, 0.01)
19    fig = plt.figure(figsize=(8, 5))
20    ax = fig.add_subplot()
21    ax.plot(tt, fun_0(tt), "b-", label="homogeneous", lw=3)
22    ax.plot(tt, fun_1(tt), "r-", label="w(t)=9")
23    ax.plot(tt, fun_2(tt), "g--", label="3exp(-3t)", lw=3)
24    ax.plot(tt, fun_3(tt), "m", label=f"3cos(3t + {phi:3.2})")
25    ax.set_xlabel("Time")
26    ax.text(4, 0.75, "y(0) = " + str(Y0))
27    ax.legend()
28    ax.set_title("First Order  $\dot{y}(t) + 3y(t) = w(t)$ ")
29
30    out = output_dir()
31    fig.savefig(out / "first_order.png", dpi=300)
32    fig.savefig(out / "first_order.pdf")
33    plt.show()
34
35
36 if __name__ == "__main__":
37     main()

```

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from ode_common import default_inputs, output_dir, solve_ode
5
6
7 def main():
8     phi = 0.0
9     we, wc = default_inputs(phi)
10    Y0 = [1, 0]
11    A = [1, 3, 2]
12

```



```

13 fun_0 = solve_ode(A=A, w=0, Y0=Y0)
14 fun_1 = solve_ode(A=A, w=1, Y0=Y0)
15 fun_2 = solve_ode(A=A, w=we, Y0=Y0)
16 fun_3 = solve_ode(A=A, w=wc, Y0=Y0)
17
18 tt = np.arange(0, 10, 0.01)
19 fig = plt.figure(figsize=(8, 5))
20 ax = fig.add_subplot()
21 ax.plot(tt, fun_0(tt), "b-", label="homogeneous", lw=3)
22 ax.plot(tt, fun_1(tt), "r-", label="w(t)=1")
23 ax.plot(tt, fun_2(tt), "g--", label="3exp(-3t)", lw=3)
24 ax.plot(tt, fun_3(tt), "m", label=f"3cos(3t + {phi:3.2})")
25 ax.text(4, 0.75, "y(0) = " + str(Y0[0]) + ", dy(0)/dt = " + str(Y0[1]))
26 ax.set_xlabel("Time")
27 ax.legend()
28 ax.set_title("Second Order:  $\ddot{y}(t) + 3\dot{y}(t) + 2y(t) = w(t)$ ")
29
30 out = output_dir()
31 fig.savefig(out / "second_order.png", dpi=300)
32 fig.savefig(out / "second_order.pdf")
33 plt.show()
34
35
36 if __name__ == "__main__":
37     main()

```

scripts/03_second_order_complex.py

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from ode_common import default_inputs, output_dir, solve_ode
5
6
7 def main():
8     phi = 0.0
9     we, wc = default_inputs(phi)
10    Y0 = [1, 0]
11    A = [1, 2, 10]
12
13    fun_0 = solve_ode(A=A, w=0, Y0=Y0)
14    fun_1 = solve_ode(A=A, w=1, Y0=Y0)
15    fun_2 = solve_ode(A=A, w=we, Y0=Y0)
16    fun_3 = solve_ode(A=A, w=wc, Y0=Y0)
17
18    tt = np.arange(0, 10, 0.01)
19    fig = plt.figure(figsize=(8, 5))
20    ax = fig.add_subplot()
21    ax.plot(tt, fun_0(tt), "b-", label="homogeneous", lw=3)
22    ax.plot(tt, fun_1(tt), "r-", label=r"$w(t)=1$")
23    ax.plot(tt, fun_2(tt), "g--", label=r"$3exp^{-3t}$", lw=3)
24    ax.plot(tt, fun_3(tt), "m", label=r"$3cos(3t + " + str(phi) + r")$")
25    ax.text(4, 0.75, "y(0) = " + str(Y0[0]) + r",  $\dot{y}(0) =$  " + str(Y0[1]))
26    ax.set_xlabel("Time")

```

```

27     ax.legend()
28     a2_str = f"{A[0]:.0f}" if A[0] != 1 else ""
29     ax.set_title(
30         rf"Second Order/Complex: ${a2_str}\ddot y(t) + {A[1]:.0f}\dot y(t) + {A[2]:.0f}y(t) = w(t)$"
31     )
32
33     out = output_dir()
34     fig.savefig(out / "second_order_complex.png", dpi=300)
35     fig.savefig(out / "second_order_complex.pdf")
36     plt.show()
37
38
39 if __name__ == "__main__":
40     main()

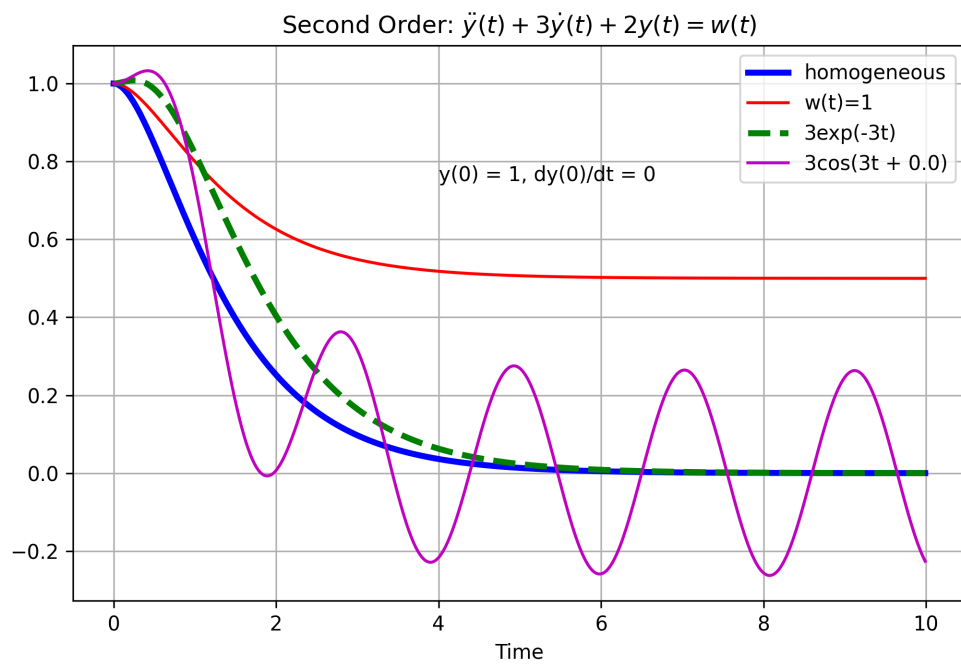
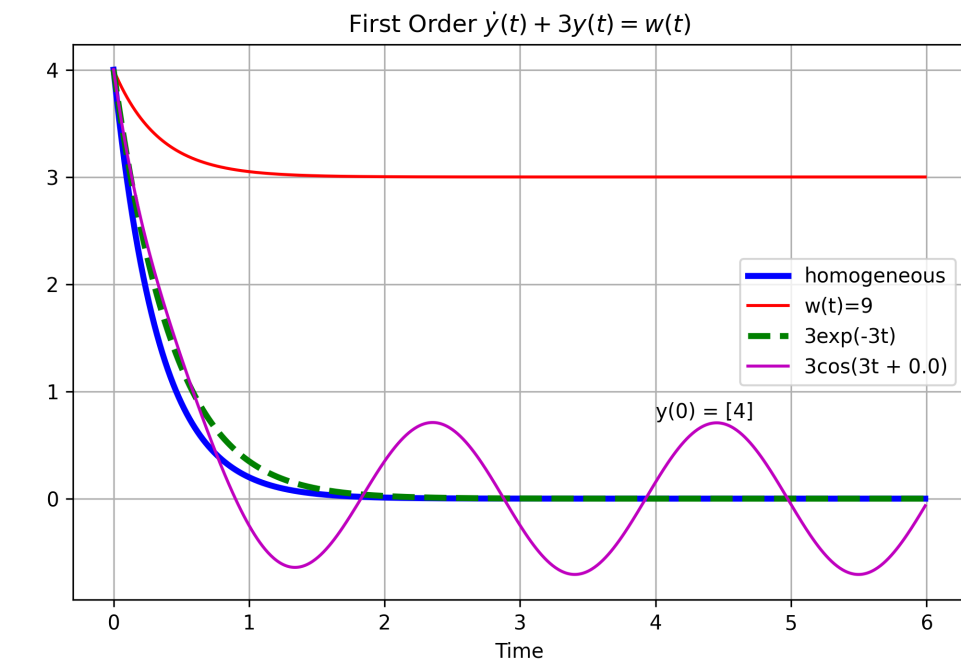
```

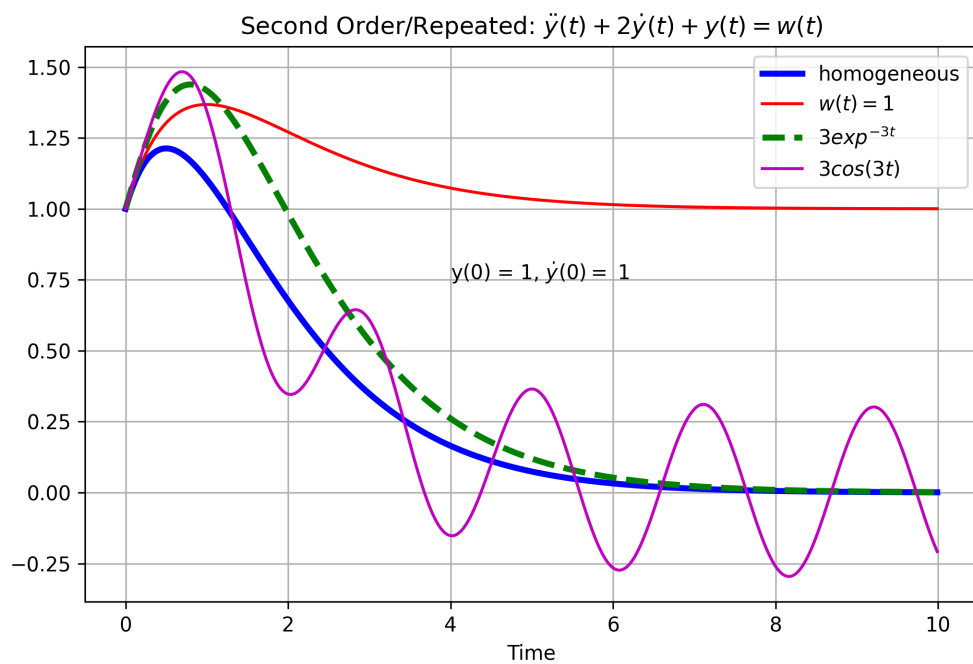
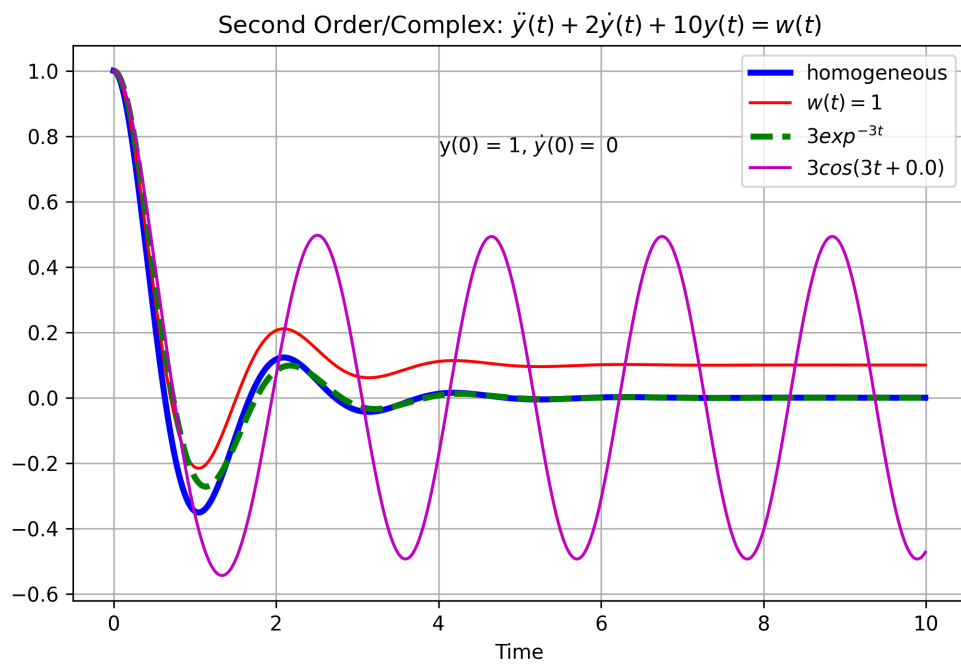
scripts/04_second_order_repeated.py

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  from ode_common import default_inputs, output_dir, solve_ode
5
6
7  def main():
8      phi = 0.0
9      we, wc = default_inputs(phi)
10     Y0 = [1, 1]
11     A = [1, 2, 1]
12
13     fun_0 = solve_ode(A=A, w=0, Y0=Y0)
14     fun_1 = solve_ode(A=A, w=1, Y0=Y0)
15     fun_2 = solve_ode(A=A, w=we, Y0=Y0)
16     fun_3 = solve_ode(A=A, w=wc, Y0=Y0)
17
18     tt = np.arange(0, 10, 0.01)
19     fig = plt.figure(figsize=(8, 5))
20     ax = fig.add_subplot()
21     ax.plot(tt, fun_0(tt), "b-", label="homogeneous", lw=3)
22     ax.plot(tt, fun_1(tt), "r-", label=r"$w(t)=1$")
23     ax.plot(tt, fun_2(tt), "g--", label=r"$3\exp^{-3t}$", lw=3)
24     ax.plot(tt, fun_3(tt), "m", label=r"$3\cos(3t)$")
25     ax.text(4, 0.75, "y(0) = " + str(Y0[0]) + r", $\dot y(0) = $" + str(Y0[1]))
26     ax.set_xlabel("Time")
27     ax.legend()
28     ax.set_title("Second Order/Repeated: $\ddot y(t) + 2\dot y(t) + y(t) = w(t)$")
29
30     out = output_dir()
31     fig.savefig(out / "second_order_repeated.png", dpi=300)
32     fig.savefig(out / "second_order_repeated.pdf")
33     plt.show()
34
35
36 if __name__ == "__main__":
37     main()

```





Numerically: The numerical comparison script is also loaded directly.

scripts/06_numerical_vs_analytical.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4
5 from ode_common import default_inputs, output_dir, solve_ode
6
7
8 def main():
9     phi = 0.0
10    we, wc = default_inputs(phi)
11
12    Y0 = [1, 0]
13    A = [1, 3, 2]
14
15    W0 = lambda t: 0.0
16    W1 = lambda t: 1.0
17    We = lambda t: 3.0 * np.exp(-3.0 * t)
18    Wc = lambda t: 3.0 * np.cos(3.0 * t)
19
20    def odes(t, Y, W):
21        y, v = Y
22        dvdt = W(t) - A[1] / A[0] * v - A[2] / A[0] * y
23        return [v, dvdt]
24
25    t_span = [0, 10]
26    t_eval = np.linspace(t_span[0], t_span[1], 100)
27
28    solution_0 = solve_ivp(odes, t_span, Y0, t_eval=t_eval, args=(W0,))
29    solution_1 = solve_ivp(odes, t_span, Y0, t_eval=t_eval, args=(W1,))
30    solution_e = solve_ivp(odes, t_span, Y0, t_eval=t_eval, args=(We,))
31    solution_c = solve_ivp(odes, t_span, Y0, t_eval=t_eval, args=(Wc,))
32
33    fun_0 = solve_ode(A=A, w=0, Y0=Y0)
34    fun_1 = solve_ode(A=A, w=1, Y0=Y0)
35    fun_2 = solve_ode(A=A, w=we, Y0=Y0)
36    fun_3 = solve_ode(A=A, w=wc, Y0=Y0)
37
38    fig = plt.figure(figsize=(8, 5))
39    ax = fig.add_subplot()
40    ax.plot(solution_0.t, solution_0.y[0], "bs", label="Numerical homogeneous", ms=4, markevery=4)
41    ax.plot(solution_1.t, solution_1.y[0], "rs", label="Numerical w(t)=1", ms=4, markevery=4)
42    ax.plot(solution_e.t, solution_e.y[0], "gs", label="Numerical 3exp(-3t)", ms=4, markevery=4)
43    ax.plot(solution_c.t, solution_c.y[0], "ms", label="Numerical 3cos(3t)", ms=4, markevery=4)
44
45    ax.plot(t_eval, fun_0(t_eval), "b-", label="Analytical homogeneous", lw=3)
46    ax.plot(t_eval, fun_1(t_eval), "r-", label=r"Analytical $w(t)=1$", lw=3)
47    ax.plot(t_eval, fun_2(t_eval), "g--", label=r"Analytical $3exp^{-3t}$", lw=3)
48    ax.plot(t_eval, fun_3(t_eval), "m", label=r"Analytical $3cos(3t)$")
49
50    ax.text(4, 0.75, "y(0) = " + str(Y0[0]) + r", $\dot{y}(0) = $" + str(Y0[1]))
51    ax.set_xlabel("Time t")
```

```

52 ax.set_ylabel("Solution")
53 ax.set_title("Second Order:  $\ddot{y}(t) + 3\dot{y}(t) + 2y(t) = w(t)$ ")
54 ax.grid(True)
55 ax.legend()
56
57 out = output_dir()
58 fig.savefig(out / "second_order_numerical_compare.png", dpi=300)
59 fig.savefig(out / "second_order_numerical_compare.pdf")
60 plt.show()
61
62
63 if __name__ == "__main__":
64     main()

```

